

Week 2 – Prototypes, Classes, Arrays: Detailed Exercises (Days 8-14)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 24 Aug 2026 – 30 Aug 2026 (Days 8-14)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm & 9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Roadmap: [ROADMAP.md](#) · Theory: [01-javascript-complete.md](#) · Index: [week-02-exercises.md](#)

Day 8 – Prototype Chain & new

Exercise 8.1: Prototype Inspection Lab

Problem statement

Create a constructor `Person(name)` that sets `this.name = name` and adds `greet()` on `Person.prototype`. Instantiate two objects and prove they share the same `greet` function but not the same `name`.

Learning objective

Understand prototype chain lookup, `[[Prototype]]`, and memory sharing vs per-instance fields.

Prerequisites

- Theory: [01-javascript-complete.md §4 Prototypes](#)

Step-by-step approach

1. Write `function Person(name) { this.name = name; }`.
2. Add `Person.prototype.greet = function() { return `Hi, ${this.name}`; }`.`
3. Create `const a = new Person("A"); const b = new Person("B");`.
4. Log `a.greet === b.greet` → should be `true`.
5. Log `a.hasOwnProperty("name")` vs `a.hasOwnProperty("greet")`.

6. Draw prototype diagram: instances → `Person.prototype` → `Object.prototype`.
7. Explain what `new` does in 4 steps (create object, link proto, bind `this`, return).

Hints

1. Methods on prototype are shared; own properties are not.
2. `Object.getPrototypeOf(a) === Person.prototype`.
3. Arrow functions on prototype still work but rare in constructor pattern.

Exact Answer

- `a.greet === b.greet` → `true` (shared prototype method)
- `a.hasOwnProperty("name")` → `true`; `a.hasOwnProperty("greet")` → `false`
- `Object.getPrototypeOf(a) === Person.prototype` → `true`

Complete Solution

```
function Person(name) {  
  this.name = name;  
}  
  
Person.prototype.greet = function () {  
  return `Hi, ${this.name}`;  
};  
  
const a = new Person("A");  
const b = new Person("B");  
  
console.log(a.greet());           // "Hi, A"  
console.log(a.greet === b.greet); // true  
console.log(a.hasOwnProperty("name")); // true  
console.log(a.hasOwnProperty("greet")); // false  
console.log(Object.getPrototypeOf(a) === Person.prototype); // true
```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(a.greet());</code>	"Hi, A"
<code>console.log(a.greet === b.greet);</code>	true
<code>console.log(a.hasOwnProperty("name"));</code>	true
<code>console.log(a.hasOwnProperty("greet"));</code>	false
<code>console.log(Object.getPrototypeOf(a) === Person.prototype);</code>	true

Expected output / acceptance criteria

- Shared `greet` reference; distinct `name` values.
- Diagram in notes or README.

Where to practice

- MDN Prototypes: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain
- Local: `mern-mastery/week-02/day-08/prototypes.js`

Common mistakes

- Putting methods inside constructor (recreated every instance).
- Confusing `__proto__` with `prototype` on functions.

80 LPA Interview Twist

Google asks you to implement `Object.create` or explain V8 hidden classes and how adding methods in the constructor vs prototype affects performance and memory at scale.

Interview connection

"How does `new` work?" and "Implement `Object.create`" test prototype literacy.

Time estimate: 50 minutes

Day 9 – ES6 Classes & Inheritance

Exercise 9.1: Animal → Dog → GuideDog Hierarchy

Problem statement

Build class hierarchy:

- `Animal : speak()` returns generic sound
- `Dog extends Animal : speak()` returns "Woof", add `fetch(item)`
- `GuideDog extends Dog : speak()` returns "Woof", add `guide(owner)`

Learning objective

Use `extends` , `super()` , method overriding, and `instanceof` checks.

Prerequisites

- Theory: [01-javascript-complete.md §4 ES6 Classes](#)

Step-by-step approach

1. Define `Animal` with constructor accepting `name` .
2. `Dog extends Animal` — call `super(name)` in constructor.
3. Override `speak` ; implement `fetch` .
4. `GuideDog extends Dog` — add `guide` without breaking chain.
5. Verify `guideDog instanceof Dog` , `instanceof Animal` .
6. Optional: static method `Animal.describe()` .
7. Note: classes are syntactic sugar over prototypes — show equivalent prototype code.

Hints

1. Must call `super()` before using `this` in subclass constructor.
2. Override methods with same name; use `super.speak()` to extend behavior.
3. `instanceof` walks prototype chain.

Exact Answer

- `gd.speak()` → "Woof"
- `gd.guide("Alex")` → e.g. "Rex is guiding Alex"
- `gd instanceof GuideDog` → `true` ; `gd instanceof Dog` → `true` ; `gd instanceof Animal` → `true`

Complete Solution

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return `${this.name} makes a sound`;
  }
}

class Dog extends Animal {
  speak() {
    return "Woof";
  }
  fetch(item) {
    return `${this.name} fetched the ${item}`;
  }
}

class GuideDog extends Dog {
  guide(owner) {
    return `${this.name} is guiding ${owner}`;
  }
}

const gd = new GuideDog("Rex");
console.log(gd.speak()); // "Woof"
console.log(gd.guide("Alex")); // "Rex is guiding Alex"
console.log(gd instanceof GuideDog); // true
console.log(gd instanceof Dog); // true
console.log(gd instanceof Animal); // true
```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(gd.speak());</code>	"Woof"
<code>console.log(gd.guide("Alex"));</code>	"Rex is guiding Alex"
<code>console.log(gd instanceof GuideDog);</code>	true
<code>console.log(gd instanceof Dog);</code>	true
<code>console.log(gd instanceof Animal);</code>	true

Expected output / acceptance criteria

```
const gd = new GuideDog("Rex");
gd.speak(); // "Woof"
gd.guide("Alex"); // meaningful string
gd instanceof GuideDog; // true
```

Where to practice

- Local: `week-02/day-09/classes.js`
- MDN Classes: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes>

Common mistakes

- Forgetting `super()` → `ReferenceError`.
- Creating circular inheritance.

80 LPA Interview Twist

LLD rounds at Amazon use this exact hierarchy pattern for Parking Lot or Zoo — expect you to diagram inheritance vs composition tradeoffs and when `instanceof` checks become a code smell.

Interview connection

OOP design in LLD (parking lot, vending machine) uses same hierarchy thinking.

Time estimate: 55 minutes

Day 10 – Objects, Destructuring, Spread

Exercise 10.1: Group By Property

Problem statement

Implement `groupBy(array, key)` returning object mapping key values to arrays of items:

```
groupBy([
  { name: "A", dept: "Eng" },
  { name: "B", dept: "HR" },
  { name: "C", dept: "Eng" },
], "dept");
// { Eng: [{...}, {...}], HR: [{...}] }
```

Learning objective

Combine objects, computed keys, and array accumulation.

Prerequisites

- Theory: [01-javascript-complete.md §6 Objects](#)

Step-by-step approach

1. Initialize `result = {}`.
2. Loop items; read `item[key]`.
3. If `!result[groupKey]`, set `result[groupKey] = []`.
4. Push item into group array.
5. Alternative: `reduce` one-liner with spread immutability variant.
6. Handle missing keys → group under `"undefined"` or skip.
7. Write 3 tests including empty array.

Hints

1. `reduce((acc, item) => { ... }, {})` is idiomatic.
2. Use optional chaining if key path nested: `key.split('.')`.
3. Immutable version copies arrays with spread on each push.

Exact Answer

- `{ Eng: [{name:"A",...}, {name:"C",...}], HR: [{name:"B",...}] }`
- `groupBy([], "dept") → {}`

Complete Solution

```
function groupBy(array, key) {
  return array.reduce((acc, item) => {
    const groupKey = item[key];
    if (!acc[groupKey]) acc[groupKey] = [];
    acc[groupKey].push(item);
    return acc;
  }, {});
}

const result = groupBy([
  { name: "A", dept: "Eng" },
  { name: "B", dept: "HR" },
  { name: "C", dept: "Eng" },
], "dept");

console.log(result);
// { Eng: [{ name: 'A', dept: 'Eng' }, { name: 'C', dept: 'Eng' }], HR: [{ name: 'B', dept: 'HR' }]}
console.log(groupBy([], "dept")); // {}
```

Expected output / acceptance criteria

- Correct grouping; order preserved within groups.
- `groupBy([], "dept") → {}`

Where to practice

- Local: `week-02/day-10/groupBy.js`
- Lodash source (read-only): <https://lodash.com/docs/#groupBy>

Common mistakes

- Mutating original array items unintentionally.
- Using `map` when `reduce` fits aggregation.

80 LPA Interview Twist

Meta backend interviews ask for `groupBy` on nested keys (`"address.city"`) with TypeScript generics and immutability — testing whether you can extend a 10-line reduce into production-grade data shaping.

Interview connection

Data transformation in API layers — grouping orders by status, users by role.

Time estimate: 40 minutes

Day 11 – Implement map, filter, reduce

Exercise 11.1: myMap, myFilter, myReduce

Problem statement

Implement without using native `.map`, `.filter`, `.reduce`:

```
function myMap(arr, fn) { /* ... */ }
function myFilter(arr, fn) { /* ... */ }
function myReduce(arr, fn, initial) { /* ... */ }
```

Learning objective

Understand callback signatures, accumulator pattern, and sparse array behavior.

Prerequisites

- Theory: [01-javascript-complete.md §5 Arrays](#)

Step-by-step approach

1. **myMap:** new array, loop `i`, push `fn(arr[i], i, arr)`.
2. **myFilter:** push when `fn(...)` truthy.
3. **myReduce:** if `initial === undefined`, use first element as acc start index 1.
4. Pass `index` and `arr` to callbacks like native APIs.
5. Test: map double, filter evens, reduce sum.
6. Compare output with native methods on same inputs.
7. Document: don't skip holes unless emulating spec exactly.

Hints

1. Reduce without initial on empty array should throw — match spec.
2. Callback args order: `(element, index, array)`.
3. Reduce accumulator is first callback arg.

Exact Answer

- `myMap([1,2,3], x => x * 2) → [2,4,6]`
- `myFilter([1,2,3,4], x => x % 2 === 0) → [2,4]`
- `myReduce([1,2,3], (a,b) => a+b, 0) → 6`

Complete Solution

```
function myMap(arr, fn) {
  const result = [];
  for (let i = 0; i < arr.length; i++) {
    result.push(fn(arr[i], i, arr));
  }
  return result;
}

function myFilter(arr, fn) {
  const result = [];
  for (let i = 0; i < arr.length; i++) {
    if (fn(arr[i], i, arr)) result.push(arr[i]);
  }
  return result;
}

function myReduce(arr, fn, initial) {
  let acc;
  let start = 0;
  if (initial === undefined) {
    if (arr.length === 0) throw new TypeError("Reduce of empty array with no initial value");
    acc = arr[0];
    start = 1;
  } else {
    acc = initial;
  }
  for (let i = start; i < arr.length; i++) {
    acc = fn(acc, arr[i], i, arr);
  }
  return acc;
}

console.log(myMap([1, 2, 3], x => x * 2)); // [2, 4, 6]
console.log(myFilter([1, 2, 3, 4], x => x % 2 === 0)); // [2, 4]
console.log(myReduce([1, 2, 3], (a, b) => a + b, 0)); // 6
```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(myMap([1, 2, 3], x => x * 2));</code>	[2, 4, 6]
<code>console.log(myFilter([1, 2, 3, 4], x => x % 2 === 0));</code>	[2, 4]
<code>console.log(myReduce([1, 2, 3], (a, b) => a + b, 0));</code>	6

Expected output / acceptance criteria

- `myMap([1,2,3], x => x * 2) → [2,4,6]`
- `myFilter([1,2,3,4], x => x % 2 === 0) → [2,4]`
- `myReduce([1,2,3], (a,b) => a+b, 0) → 6`

Where to practice

- Local: `week-02/day-11/array-methods.js`
- You Don't Know JS (types): <https://github.com/getify/You-Dont-Know-JS>

Common mistakes

- Wrong reduce initial causing off-by-one.
- Forgetting to return accumulator in reduce callback.

80 LPA Interview Twist

Direct ask at Google: "Implement lazy map/filter chain (Iterable)" or polyfill with sparse-array semantics — they want spec-accurate behavior, not just happy-path loops.

Interview connection

Direct interview ask: "Implement Array.prototype.map" — tests loops + callbacks.

Time estimate: 75 minutes

Day 12 – LC 88 Merge Sorted Array

Exercise 12.1: LC 88 – Merge Sorted Array

Problem statement

Merge `nums2` into `nums1` in-place; `nums1` has length `m + n` with first `m` elements valid.

Learning objective

Merge from end with two pointers to avoid overwriting unmerged elements.

Prerequisites

- Two pointers intro: [08-dsa-patterns-complete.md §1](#)

Step-by-step approach

1. Start pointers: `i = m-1`, `j = n-1`, `write = m+n-1`.
2. Compare from end; place larger at `nums1[write]`, decrement pointers.
3. Copy remaining `nums2` if any.

4. Avoid sorting merged result $O(n \log n)$ penalty.
5. Trace `[1,2,3,0,0,0]` + `[2,5,6]` on paper.

Hints

1. Merging from front needs extra buffer; from back is in-place.
2. When `nums1` elements remain, they're already in place.
3. `while (j >= 0)` handles `nums2` leftovers.

Exact Answer

- After merge: `[1,2,2,3,5,6]`
- Time: $O(m+n)$, Space: $O(1)$

Complete Solution

```
function merge(nums1, m, nums2, n) {
  let i = m - 1;
  let j = n - 1;
  let write = m + n - 1;

  while (j >= 0) {
    if (i >= 0 && nums1[i] > nums2[j]) {
      nums1[write--] = nums1[i--];
    } else {
      nums1[write--] = nums2[j--];
    }
  }
}

const nums1 = [1, 2, 3, 0, 0, 0];
merge(nums1, 3, [2, 5, 6], 3);
console.log(nums1); // [1, 2, 2, 3, 5, 6]
```

Expected output / acceptance criteria

- After merge: `[1,2,2,3,5,6]`

Where to practice

- <https://leetcode.com/problems/merge-sorted-array/>

Common mistakes

- Merging forward and overwriting `nums1` values not yet compared.
- Ignoring that `m / n` may be zero.

80 LPA Interview Twist

Follow-up: Merge k sorted arrays (LC 23) with a min-heap — Amazon uses LC 88 as a stepping

stone to assess whether you recognize the merge pattern generalizes.

Interview connection

Merge k sorted lists and merge intervals extend two-pointer merge.

Time estimate: 35 minutes

Day 13 – Flatten, Curry & LC 27, 977

Exercise 13.1: Deep Flatten Array

Problem statement

`flatten(arr)` → single-level array from arbitrary nesting:

`flatten([1, [2, [3, [4]], 5]])` → `[1, 2, 3, 4, 5]`

Learning objective

Recursion vs iterative stack; distinguish depth-limited vs full flatten.

Prerequisites

- Theory: [01-javascript-complete.md §5](#)

Step-by-step approach

1. Loop elements; if `Array.isArray(el)`, concat recurse result.
2. Else push to result.
3. Test depth 5+ nested array.
4. Optional: `flatten(arr, depth)` using `flat` spec behavior.
5. Iterative: use stack of items to process.
6. Analyze time $O(n)$ total elements, space $O(\text{depth})$.

Hints

1. `Array.isArray` is safer than `typeof`.
2. Spread or `concat` to merge recursive results.
3. `arr.flat(Infinity)` exists — implement manually first.

Exact Answer

- `flatten([1, [2, [3, [4]], 5]])` → `[1, 2, 3, 4, 5]`
- Time: $O(n)$ total elements, Space: $O(\text{depth})$ recursion stack

Complete Solution

```
function flatten(arr) {  
  const result = [];  
  for (const el of arr) {  
    if (Array.isArray(el)) {  
      result.push(...flatten(el));  
    } else {  
      result.push(el);  
    }  
  }  
  return result;  
}  
  
console.log(flatten([1, [2, [3, [4]], 5]])); // [1, 2, 3, 4, 5]  
console.log(flatten([1, [2, [3]]]));         // [1, 2, 3]
```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(flatten([1, [2, [3, [4]], 5]]));</code>	[1, 2, 3, 4, 5]
<code>console.log(flatten([1, [2, [3]]]));</code>	[1, 2, 3]

Expected output / acceptance criteria

- Handles empty arrays and mixed types.
- `[1, [2, [3]]] → [1, 2, 3]`

Where to practice

- Local: `week-02/day-13/flatten.js`
- LC 131 Flatten (premium concept): similar recursion

Common mistakes

- Infinite recursion on circular refs (don't assume for basic exercise).
- Only flattening one level.

80 LPA Interview Twist

Meta asks for iterative stack-based flatten with depth limit, or handling circular references — production JSON normalizers need these edge cases.

Interview connection

Tree/array flattening appears in nested JSON API normalization.

Time estimate: 45 minutes

Exercise 13.2: Curry Function

Problem statement

Implement `curry(fn)` so:

```
const add = curry((a, b, c) => a + b + c);  
add(1)(2)(3); // 6  
add(1, 2)(3); // 6  
add(1, 2, 3); // 6
```

Learning objective

Partial application, arity tracking, closure accumulation.

Prerequisites

- Theory: [01-javascript-complete.md §2 Closures](#)

Step-by-step approach

1. `curry(fn)` returns function `curried(...args)`.
2. If `args.length >= fn.length`, invoke `fn(...args)`.
3. Else return new function collecting more args: `(...next) => curried(...args, ...next)`.
4. Test unary, binary, ternary functions.
5. Edge: rest parameters `fn.length` may be 0 — document limitation.
6. Compare with `bind` partial application.

Hints

1. `Function.length` is declared param count before rest.
2. Spread accumulated args on each call.
3. Return value vs return function depends on arity.

Exact Answer

- `add(1)(2)(3) → 6`
- `add(1, 2)(3) → 6`
- `add(1, 2, 3) → 6`

Complete Solution

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...next) => curried(...args, ...next);
  };
}

const add = curry((a, b, c) => a + b + c);
console.log(add(1)(2)(3)); // 6
console.log(add(1, 2)(3)); // 6
console.log(add(1, 2, 3)); // 6

const add2 = curry((a, b) => a + b);
console.log(add2(1)(2)); // 3
```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(add(1)(2)(3));</code>	6
<code>console.log(add(1, 2)(3));</code>	6
<code>console.log(add(1, 2, 3));</code>	6
<code>console.log(add2(1)(2));</code>	3

Expected output / acceptance criteria

- All three `add` call styles return 6.
- Works for `curry((a,b) => a+b)(1)(2)` .

Where to practice

- Local: `week-02/day-13/curry.js`
- Ramda curry docs: <https://ramdajs.com/docs/#curry>

Common mistakes

- Not handling multi-arg first call `add(1,2)(3)` .
- Relying on `fn.length` with default/rest parameters incorrectly.

80 LPA Interview Twist

Google frontend asks for curry with placeholder support (`_`) like Ramda, or uncurry — tests deep functional programming literacy beyond the basic arity check.

Interview connection

Classic frontend interview question; used in functional API design (Redux curried actions).

Time estimate: 60 minutes

Exercise 13.3: LC 27 – Move Zeroes

Problem statement

Move all 0s to end in-place; maintain relative order of non-zero elements.

Learning objective

Two pointers — write index for next non-zero placement.

Step-by-step approach

1. `let write = 0` ; scan with `read` .
2. If `nums[read] !== 0` , swap or assign to `nums[write++]` .
3. Second pass fill zeros optional if using swap-only approach.
4. Single-pass variant: swap non-zero forward.

Hints

1. "Snowball" technique accumulates zeros count.
2. Relative order preserved by sequential write index.
3. In-place means $O(1)$ extra space.

Exact Answer

- `[0,1,0,3,12] → [1,3,12,0,0]`
- Time: $O(n)$, Space: $O(1)$

Complete Solution

```
function moveZeroes(nums) {  
  let write = 0;  
  for (let read = 0; read < nums.length; read++) {  
    if (nums[read] !== 0) {  
      [nums[write], nums[read]] = [nums[read], nums[write]];  
      write++;  
    }  
  }  
}  
  
const nums = [0, 1, 0, 3, 12];  
moveZeroes(nums);  
console.log(nums); // [1, 3, 12, 0, 0]
```

Expected output / acceptance criteria

- `[0,1,0,3,12] → [1,3,12,0,0]`

Where to practice

- <https://leetcode.com/problems/move-zeroes/>

Common mistakes

- Using extra array (not in-place).
- Losing order with sort.

80 LPA Interview Twist

Variant LC 75 Sort Colors (Dutch flag) requires three-way partition — same pointer technique but interviewers expect $O(n)$ one-pass with 0/1/2 ordering proof.

Interview connection

Partition patterns (LC 75 Sort Colors) use same pointer technique.

Time estimate: 25 minutes

Exercise 13.4: LC 977 – Squares of Sorted Array

Problem statement

Return squares of sorted array, also sorted: `[-4,-1,0,3,10] → [0,1,9,16,100]`.

Learning objective

Two pointers from ends — larger square comes from larger absolute value.

Step-by-step approach

1. `left = 0` , `right = n-1` , `result = new Array(n)` .
2. Fill from end: compare `abs(nums[left])` vs `abs(nums[right])` .
3. Place larger square at `result[k--]` .
4. $O(n)$ time, $O(n)$ output space.

Hints

1. Negative numbers square large — two pointers avoid sort.
2. Fill result backwards so you don't overwrite.
3. Handle all-negative or all-positive arrays.

Exact Answer

- `[-4,-1,0,3,10] → [0,1,9,16,100]`
- Time: $O(n)$, Space: $O(n)$ for result array

Complete Solution

```
function sortedSquares(nums) {
  let left = 0;
  let right = nums.length - 1;
  const result = new Array(nums.length);
  let k = nums.length - 1;

  while (left <= right) {
    const leftSq = nums[left] * nums[left];
    const rightSq = nums[right] * nums[right];
    if (leftSq > rightSq) {
      result[k--] = leftSq;
      left++;
    } else {
      result[k--] = rightSq;
      right--;
    }
  }
  return result;
}

console.log(sortedSquares([-4, -1, 0, 3, 10])); // [0, 1, 9, 16, 100]
```

Expected output / acceptance criteria

- Correct sorted squares for all test cases.

Where to practice

- <https://leetcode.com/problems/squares-of-a-sorted-array/>

Common mistakes

- Sorting after squaring $O(n \log n)$ when $O(n)$ requested in interviews.

80 LPA Interview Twist

Two-pointer merge pattern from both ends also applies to LC 977 follow-ups and merge sorted arrays — expect "why not sort?" with complexity justification in one sentence.

Interview connection

Merge sorted arrays from two ends — symmetric two-pointer pattern.

Time estimate: 30 minutes

Day 14 – Week 2 Review & LC 6, 9, 26

Exercise 14.1: LC 6 – Product of Array Except Self

Problem statement

Return array where `output[i]` = product of all elements except `nums[i]`. $O(n)$ time, no division.

Learning objective

Prefix and suffix products in two passes.

Step-by-step approach

1. Pass left→right: `output[i] = product of nums[0..i-1]`.
2. Pass right→left: multiply by suffix product using running variable.
3. Or build prefix array and suffix array explicitly.
4. Explain why division fails (zeros, integer overflow discussion).

Hints

1. Initialize `output` with 1s or use left products in first pass.
2. Running `suffix` variable avoids second array.
3. Single element array edge case.

Exact Answer

- `[1,2,3,4] → [24,12,8,6]`
- Time: $O(n)$, Space: $O(1)$ excluding output (two-pass with running suffix)

Complete Solution

```
function productExceptSelf(nums) {  
  const n = nums.length;  
  const output = new Array(n).fill(1);  
  
  let prefix = 1;  
  for (let i = 0; i < n; i++) {  
    output[i] = prefix;  
    prefix *= nums[i];  
  }  
  
  let suffix = 1;  
  for (let i = n - 1; i >= 0; i--) {  
    output[i] *= suffix;  
    suffix *= nums[i];  
  }  
  return output;  
}  
  
console.log(productExceptSelf([1, 2, 3, 4])); // [24, 12, 8, 6]
```

Expected output / acceptance criteria

- `[1,2,3,4] → [24,12,8,6]`

Where to practice

- <https://leetcode.com/problems/product-of-array-except-self/>

Common mistakes

- Using division — forbidden and breaks on zero.
- $O(n^2)$ nested loops on medium problem.

80 LPA Interview Twist

Follow-up at Google: product with zero handling without extra pass, or product modulo 10^9+7 — tests whether you understand prefix/suffix as a general technique (trapping rain water).

Interview connection

Prefix/suffix technique recurs in trapping rain water and stock problems.

Time estimate: 45 minutes

Exercise 14.2: LC 9 – Palindrome Number

Problem statement

Return true if integer reads same backward.

Learning objective

Reverse half the digits in-place without string conversion (or justify string approach).

Step-by-step approach

1. Negative $\rightarrow$ false; ending in 0 (except 0) $\rightarrow$ false.
2. Reverse second half: while `x > reversed`, `reversed = reversed*10 + x%10`, `x = floor(x/10)`.
3. Compare `x === reversed` or `x === floor(reversed/10)` for even/odd digits.

Hints

1. String reverse is acceptable in JS interviews if you mention tradeoffs.
2. Modulo extracts last digit.
3. Stop when original $\leq$ reversed — processed half.

Exact Answer

- `121  $\rightarrow$  true`; `123  $\rightarrow$  false`; `-121  $\rightarrow$  false`
- Time: $O(\log n)$, Space: $O(1)$ (half-digit reversal)

Complete Solution

```
function isPalindrome(x) {
  if (x < 0 || (x !== 0 && x % 10 === 0)) return false;
  let reversed = 0;
  while (x > reversed) {
    reversed = reversed * 10 + x % 10;
    x = Math.floor(x / 10);
  }
  return x === reversed || x === Math.floor(reversed / 10);
}

console.log(isPalindrome(121)); // true
console.log(isPalindrome(123)); // false
console.log(isPalindrome(-121)); // false
```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(isPalindrome(121));</code>	true
<code>console.log(isPalindrome(123));</code>	false
<code>console.log(isPalindrome(-121));</code>	false

Expected output / acceptance criteria

- `121` → true; `123` → false

Where to practice

- <https://leetcode.com/problems/palindrome-number/>

Common mistakes

- Integer overflow (less issue in JS doubles).
- Converting to string without noting $O(\log n)$ extra space.

80 LPA Interview Twist

In Java/C++ ports they ask about integer overflow on reversal — at Meta you should mention JS Number is safe to $2^{53}-1$ and discuss BigInt for financial palindrome checks.

Interview connection

Digit manipulation and overflow discussions in Java/C++ ports of same problem.

Time estimate: 25 minutes

Exercise 14.3: LC 26 – Remove Duplicates from Sorted Array

Problem statement

Remove duplicates in-place from sorted array; return new length.

Learning objective

Slow/fast pointer on sorted array — duplicates are adjacent.

Step-by-step approach

1. If length 0 return 0.
2. `write = 1`; for `read` from 1 to end, if `nums[read] !== nums[read-1]`, assign to `nums[write++]`.
3. First `write` elements are unique.

Hints

1. Sorted → duplicates are consecutive.
2. Don't slice array — return length only.
3. Minimum length 1 always returns 1.

Exact Answer

- `[1,1,2]` → length `2`, nums begins `[1,2,...]`
- Time: $O(n)$, Space: $O(1)$

Complete Solution

```
function removeDuplicates(nums) {  
  if (nums.length === 0) return 0;  
  let write = 1;  
  for (let read = 1; read < nums.length; read++) {  
    if (nums[read] !== nums[read - 1]) {  
      nums[write++] = nums[read];  
    }  
  }  
  return write;  
}  
  
const nums = [1, 1, 2];  
const k = removeDuplicates(nums);  
console.log(k, nums.slice(0, k)); // 2 [1, 2]
```

Expected output / acceptance criteria

- `[1,1,2]` → length 2, nums begins `[1,2,...]`

Where to practice

- <https://leetcode.com/problems/remove-duplicates-from-sorted-array/>

Common mistakes

- Using Set (not in-place requirement).
- Returning array instead of k.

80 LPA Interview Twist

LC 80 (allow 2 duplicates) and LC 26 in-place requirement often combined — Amazon asks both in one 20-minute slot with pointer invariant explained aloud.

Interview connection

Remove duplicates II (allow 2 copies) — pointer variant.

Time estimate: 20 minutes

Exercise 14.4: Week 2 Personal Cheat Sheet

Problem statement

Write `week-02/CHEATSHEET.md` in your own words: prototypes vs classes, map/filter/reduce signatures, curry use case, two-pointer when to use.

Learning objective

Consolidate Week 2 into interview-ready summaries.

Step-by-step approach

1. One page max per topic with code snippet.
2. Include diagram for prototype chain.
3. List 3 problems where two pointers apply.
4. Commit to GitHub `mern-mastery/week-02`.

Exact Answer

- One-page cheat sheet covering: prototype chain diagram, map/filter/reduce signatures, curry use case, two-pointer problems list
- Self-test: explain `new` in 4 steps, implement curry from memory

Complete Solution

```
# Week 2 Cheat Sheet (sample structure)

## Prototypes vs Classes
- `new`: create obj → link proto → bind this → return (unless object returned)
- Methods on `Constructor.prototype` are shared

## Array Methods
- map(fn): (el, i, arr) => new array
- filter(fn): truthy predicate → subset
- reduce(fn, init): (acc, el, i, arr) => single value

## Curry
- Partial application via closure; `fn.length` tracks arity

## Two Pointers
- LC 88 merge from end, LC 27 move zeroes, LC 977 squares from ends, LC 26 dedupe
```

Expected output / acceptance criteria

- Cheat sheet committed; all Week 2 exercises pass self-test.

80 LPA Interview Twist

At Google/Meta, cheat sheets become "teach-back" rounds — interviewer picks any Week 2 topic and asks you to whiteboard it in 5 minutes without notes; memorization alone fails.

Where to practice

- Local repo: [mern-mastery/week-02/](#)

Time estimate: 2 hours

Week 2 Completion Checklist

Item	Done
Prototype lab + class hierarchy	☐
myMap/myFilter/myReduce	☐
flatten + curry	☐
groupBy	☐
LC 6, 88, 27, 977, 9, 26	☐
Code in week-02/	☐

Total estimated time: ~20–24 hours